# WAITLENS: Temporal VLM-Based Adaptive Waiting for Industrial GUI Testing Agents

Shika Ikeura*, Xuan Wang*, Ziyi Kuang*, Jianping Xu*, Chao Kang†,
Yongxiang Hu*, Xiaoyu Wang†, Juxing Yuan†, and Yangfan Zhou*
*College of Computer Science and Artificial Intelligence, Fudan University, Shanghai, China
†Meituan, Beijing, China
{pcgchi25, xuanwang25, zykuang25, jpxu25}@m.fudan.edu.cn, kangchao@meituan.com,
huyx@fudan.edu.cn, {wangxiaoyu17, yuanjuxing}@meituan.com, zyf@fudan.edu.cn

***Abstract*—Improving the efficiency of GUI testing agents is important for industrial deployment. Beyond LLM inference, fixed waiting between consecutive interactions introduces substantial overhead. Our industrial study reveals that fixed waiting accounts for 42.3% of the time within the sampled interaction intervals, comparable to the 54.8% spent on model calls. Moreover, 62.9% of the waiting time is spent after the GUI has already finished loading. Although prior studies have proposed adaptive waiting strategies, their reliance on app-specific data or platform-specific information limits their applicability across diverse industrial environments. To this end, we propose WAITLENS, which fine-tunes a lightweight VLM on GUI loading trajectories annotated with waiting end times. We introduce temporal resampling to construct diverse training examples, enabling the model to learn varying GUI loading behaviors from temporal observations. Experiments show that WAITLENS achieves 86.08% accuracy in deciding whether to keep waiting, compared with 54.13% for the backbone VLM. Our evaluation using recordings from Meituan's industrial testing pipeline shows that WAITLENS reduces mean extra waiting after the interface is ready by 94.98% compared with a 20 s fixed wait.**

***Index Terms*—GUI Testing Agents, Adaptive Waiting, GUI Rendering, Vision-Language Models**

## I. INTRODUCTION

Recently, large language model (LLM) based GUI agents have emerged as a new testing method, which reduces reliance on manually maintained test scripts [1]–[8]. Given natural language testing tasks, these agents repeatedly observe GUI screenshots, reason about subsequent actions, and interact with apps to complete testing tasks. As illustrated in Figure 1, each action is followed by a waiting period before the agent captures the next screenshot, allowing the interface to respond.

Determining an appropriate waiting time, however, is challenging. If the wait is too short, agents may mistake an intermediate loading state for the result of the preceding action, leading to incorrect decisions [9], [10]. To accommodate slow responses, existing GUI agents often adopt fixed waiting times after each action [11]–[13]. However, a long fixed wait also keeps agents idle when the interface becomes ready earlier. Our motivation study in Section II-B shows that 62.9% of the waiting time in our industrial GUI testing agent is spent idling after the GUI has already finished loading. This substantial overhead motivates adaptive waiting policies in GUI testing agents.

Although prior studies have explored adaptive waiting for GUI testing [14]–[17], these methods face limitations when applied to GUI agents across diverse industrial environments. First, learning-based methods rely on app-specific training data and may struggle with unseen GUIs. For example, AdaT [14] uses a convolutional neural network (CNN) to classify GUI rendering states. To adapt AdaT to WeChat, WeReplay [15] collects new training data from the app and fine-tunes the model. Second, analysis-based methods rely on internal runtime or platform-specific information that screenshot-based agents cannot directly access. For example, Zhang et al. [16] monitor DOM changes, while TurboDroid [17] uses method dependency analysis to determine waiting conditions. Therefore, GUI agents need an adaptive waiting approach that relies only on screenshots and can generalize across different GUIs.

Vision-language models (VLMs) have demonstrated strong capabilities in understanding diverse GUIs [18]–[21], making them a promising alternative. However, directly applying existing VLMs to adaptive waiting is impractical for two reasons. First, unlike static GUI tasks, judging whether an interface has finished loading requires observing how the screen changes over time [22], [23]. Simply feeding all historical screenshots into a VLM increases input size and inference latency as loading continues. This may offset the time saved by adaptive waiting. Second, general-purpose VLMs are not specifically trained to distinguish loading states from completed interfaces. As a result, they may struggle to accurately predict whether the GUI is ready.

To address these challenges, we propose WAITLENS, an VLM-based approach for adaptive waiting in GUI testing agents. Instead of waiting for a fixed duration, WAITLENS periodically checks the screen after each action to decide when testing should proceed. To capture interface changes within a bounded input size, WAITLENS uses a sliding window of screenshots as input to model. During VLM fine-tuning, we use temporal resampling to augment the training data and simulate different GUI loading speeds. Together, these designs enable WAITLENS to predict GUI readiness across diverse loading conditions with low inference overhead.

We conduct comprehensive experiments to evaluate the effectiveness of WAITLENS. Our in-house experiments show that WAITLENS achieves 86.08% accuracy in GUI readiness

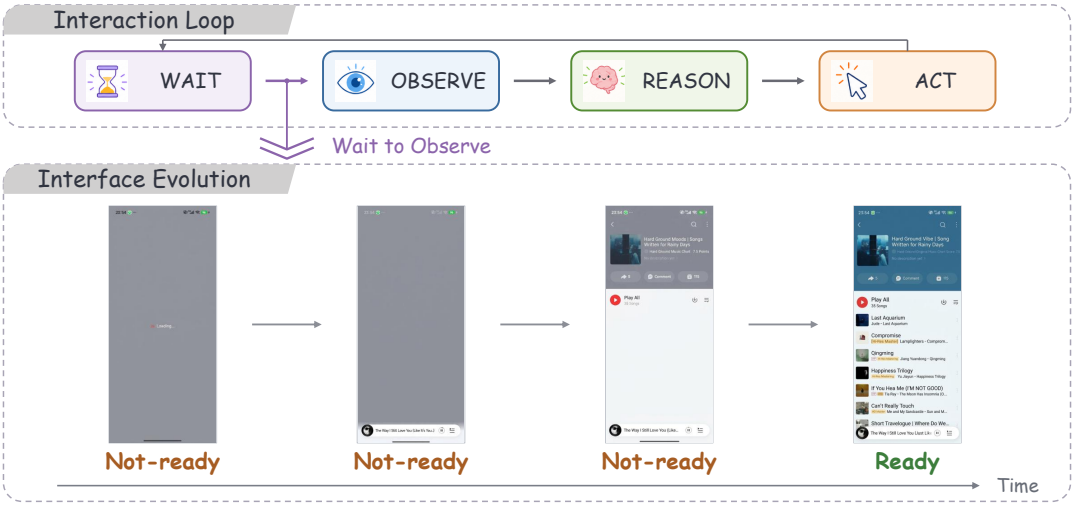



Fig. 1. GUI testing agent interaction loop and interface evolution during post-action waiting. The upper panel shows the interaction loop, and the lower panel illustrates the interface progressing from intermediate states to Ready.

prediction, compared with 54.13% for the backbone VLM Qwen3-VL-2B-Instruct [24]. Moreover, we evaluate WAITLENS using industrial GUI testing recordings from Meituan, a major Chinese local-services platform with over 800 million transacting users [25]. In our industrial replay evaluation, WAITLENS reduces mean post-Ready waiting by 94.98% compared with the existing fixed waiting strategy. We further analyze representative failure cases to understand the remaining challenges and identify directions for future research.

We summarize the contributions of this paper as follows.

- We share industrial observations that identify fixed post-action waiting as a primary source of overhead in GUI testing agents. Our motivation study reveals that over 60% of the fixed waiting time is wasted after the interface is already ready.
- We introduce WAITLENS, an VLM-based adaptive waiting approach that learns when to end waiting from temporal GUI observations. We make this approach practical by fine-tuning a lightweight open-source VLM on annotated GUI loading sequences.
- Comprehensive experiments show that WAITLENS outperforms existing baselines in detecting GUI loading completion with low runtime overhead. Moreover, our evaluation on Meituan's industrial recordings demonstrates its practical potential for reducing unnecessary waiting, while failure case analysis provides insights for future research.

## II. BACKGROUND

### A. Industrial Practice

Industrial mobile GUI testing has traditionally relied on automated exploration [26]–[28] and manually maintained test scripts [29], [30]. More recently, LLM-based GUI testing agents have been introduced to execute test tasks described in natural language [4], [6]. In industrial regression testing, these agents may run across diverse devices and network conditions to cover different user environments [31] (*e.g.*, AWS Device Farm [32] and Firebase Test Lab [33]). Differences in hardware capabilities and network conditions can lead to substantial variations in GUI loading time, ranging from less than a second to tens of seconds. Unlike random exploration tools, GUI testing agents rely on the outcome of each interaction to plan subsequent actions. Observing the screen too early risks capturing blank screens, loading spinners, or other intermediate loading states that mislead the agent into incorrect actions. To avoid such errors, existing GUI agent implementations often adopt configurable fixed waiting times [11], [12], [34]–[36].

At Meituan, we deploy GUI testing agents [4], [8] to verify mobile services including food delivery, hotel booking, and retail. Our testing pipeline executes tasks across a shared pool of physical devices, including resource-constrained smartphones. To accommodate slow-loading interfaces, the configuration examined in this study adopts a fixed 20-second waiting interval after each interaction before the agent captures its next screenshot. To investigate the overhead introduced by this waiting strategy, we analyze real-world GUI agent executions from Meituan in the following subsection.

### B. Motivation Study

To quantify the waiting overhead in industrial GUI testing agents, we investigate the following question: *How much execution time do GUI testing agents spend on fixed waiting, and how much of that waiting is unnecessary?*

We collect 100 screen recordings of GUI testing agent executions from Meituan's regression pipeline, covering four low-end devices under slow network conditions. The pipeline adopts a fixed waiting interval of $W = 20$ seconds after each interaction. Following prior work [14], we focus on click interactions, yielding 468 clicks. We regard an interface as

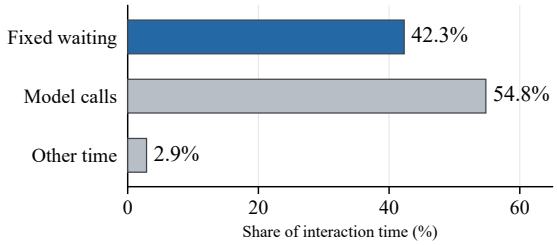



Fig. 2. Breakdown of time within the sampled click interaction intervals.

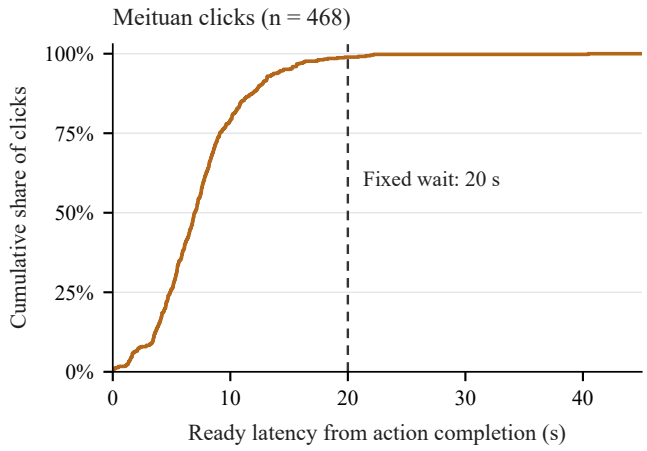



Fig. 3. Empirical cumulative distribution of Ready latency for 468 Meituan clicks. The dashed line marks the configured 20-second waiting interval.

*Ready* when the primary response to the preceding action has appeared and provides sufficient information for the agent's next observation. For each click, two annotators independently identify the earliest time when the interface reaches the Ready state, with disagreements resolved by a third reviewer.

We measure two complementary metrics: *Ready latency* and *post-Ready waiting*. Specifically, we break down the time within the sampled interaction intervals into fixed waiting, model calls, and other time, including action execution. Ready latency measures the elapsed time from action completion to the moment the interface reaches Ready. It reflects how quickly the interface becomes sufficient for the agent's next observation. Post-Ready waiting measures the time between reaching Ready and the end of the 20-second fixed waiting interval. It is zero when Ready latency is at least 20 seconds. A higher value indicates more redundant waiting after the interface is sufficient for the agent's next observation.

Figure 2 presents the runtime breakdown. Fixed waiting accounts for 42.3% of the measured execution time, comparable to the 54.8% spent on model calls, while other time accounts for 2.9%. This indicates that waiting remains a substantial source of execution overhead in LLM-based GUI testing agents.

Figure 3 shows the distribution of Ready latency. The median latency is 6.97 seconds, substantially shorter than the configured 20-second wait. At the same time, response times vary widely across interactions. Overall, 463 out of 468 clicks (98.9%) reach Ready within 20 seconds, while the remaining five require longer. Consequently, post-Ready waiting averages 12.58 seconds per click, accounting for 62.9% of the total fixed waiting time. Across all 468 clicks, the configured waits total 156 minutes, of which approximately 98 minutes are spent idling after the interface is already sufficient for the agent's next decision. This reveals substantial unnecessary waiting in the existing industrial testing pipeline.

These findings highlight the limitations of fixed waiting. Shorter intervals reduce redundant waiting but increase the risk of premature observations, while longer intervals accommodate slow responses at the cost of additional waiting. Therefore, adaptive waiting is needed to balance testing effectiveness and efficiency across diverse GUI interactions.

## III. Method

### A. Overview

WaitLens is a VLM-based adaptive waiting plugin for GUI testing agents. It replaces the fixed waiting time after each interaction by determining when the interface is ready for the next observation.

As shown in Figure 4 (right), WaitLens is invoked immediately after the agent completes an action. During online waiting, WaitLens periodically captures the latest screenshot and combines it with the pre-interaction screenshot and a limited number of recent historical screenshots. The trained VLM uses this temporal visual input to predict whether the GUI has reached the Ready state. Waiting ends early once the model produces the required number of consecutive `ready` predictions. WaitLens then returns the latest checked screenshot to the agent for its next observation. Otherwise, it continues checking subsequent screenshots. If the original waiting budget expires before this condition is met, WaitLens returns the latest available screenshot.

To train the VLM, we construct labeled visual sequences from annotated interaction recordings and augment them through temporal resampling. We then fine-tune a lightweight pretrained VLM (Qwen3-VL-2B) using these samples, as illustrated in Figure 4 (left).

### B. Bounded Temporal Visual Representation

A single screenshot may not reveal whether the GUI has reached the Ready state, as the GUI evolves over time. For example, Figure 5 shows that the playlist header appears at 1.25 s, while the song list does not appear until 1.50 s. Without observing this change, the earlier screenshot may be mistakenly considered Ready. Therefore, we organize the VLM input using three complementary types of screenshots to capture GUI changes over time. The pre-interaction screenshot provides a reference for identifying changes triggered by the action. Historical screenshots capture how the interface evolves, while the current screenshot is used to determine whether waiting should end.

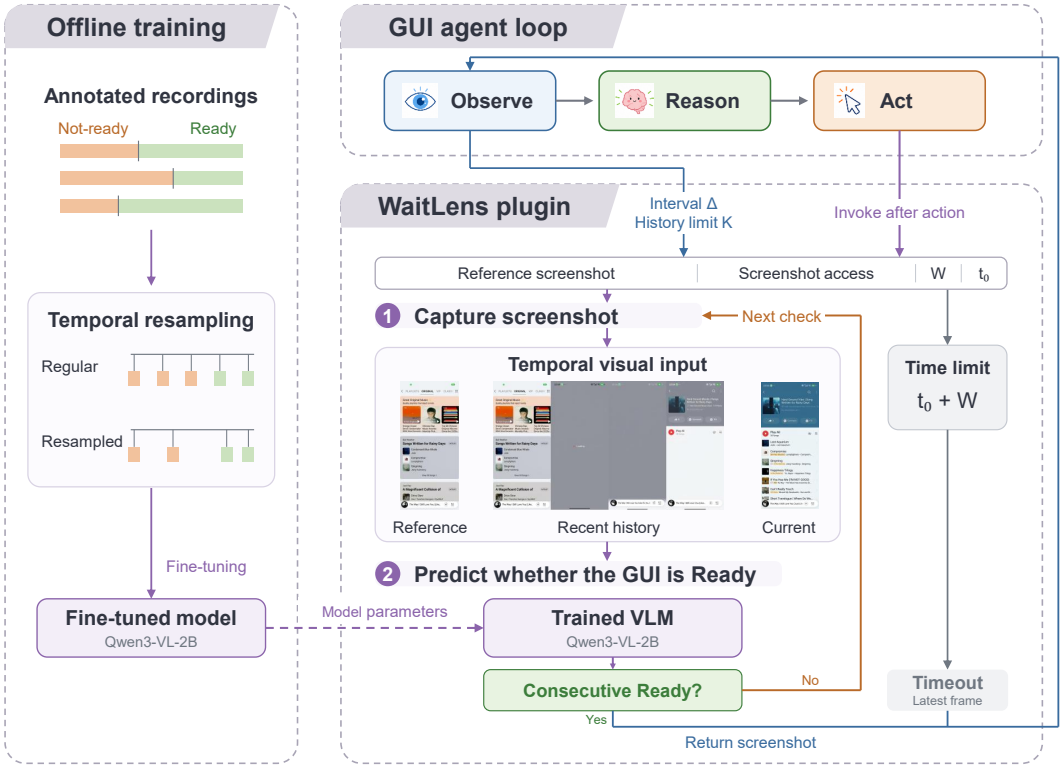



Fig. 4. Overview of WAITLENS. Online, the VLM uses bounded temporal visual input to predict whether the GUI has reached the Ready state and control waiting. Regularly sampled and temporally resampled examples are used to train the VLM.

Historical screenshots help the VLM identify recent GUI changes, but retaining every screenshot would increase the input size as waiting continues. To expose recent interface changes while keeping the input size bounded, we select historical screenshots using a sliding window. Specifically, let $t_0$ denote action completion, $\Delta$ the screenshot sampling interval, and $K$ the maximum history length. Screenshots are sampled at regular intervals $t_k = t_0 + k\Delta$ ($k \geq 1$). For the current observation at $t_k$, the history retains at most the $K$ most recent preceding screenshots:

$$H_k = [I(t_{k-N}), \ldots, I(t_{k-1})], \qquad N = \min(K, k-1), \quad (1)$$

where $I(t)$ denotes the screenshot at time $t$. Thus, the history grows as early screenshots become available ($k \leq K$) and slides forward to drop older frames once it reaches the limit of $K$ images. In the example of Figure 5, the sampling interval is $\Delta = 0.25$ s and the history limit is $K = 4$.

After selecting the history, we concatenate its images from oldest to newest into a horizontal strip. The pre-interaction and current screenshots remain separate, preserving the distinction between contextual evidence and the image being judged. To preserve interface details while limiting input size, we retain the pre-interaction and current screenshots at higher resolutions and downsample the historical screenshots. When no history exists ($N = 0$), the history strip is omitted. Finally, we provide these images to the VLM with a prompt specifying their roles and temporal order. The VLM predicts whether the current interface is `ready` or `not_ready`.

### C. VLM Fine-Tuning with Temporal Resampling

To train the VLM to predict when waiting should end, we construct labeled examples from annotated action recordings $S$. For each recording, let $t_0$ denote action completion, $t_R$ the annotated earliest Ready time, and $t_E$ the start of the next action or the end of the recording. Observations before $t_R$ are labeled `not_ready`, while those from $t_R$ to $t_E$ are labeled `ready`. Following the online procedure described in Section III-B, we sample screenshots at regular intervals of $\Delta$ and pair each current screenshot with the pre-interaction reference and up to $K$ preceding screenshots. To avoid over-representing slow-loading actions, we retain at most one regularly sampled example per label for each action, forming the baseline set $D_G$.

However, in $D_G$, fixed-interval sampling may introduce a spurious correlation between history length and training labels. Since history length increases with elapsed time until reaching $K$, early observations tend to have shorter histories than later ones. As GUIs typically become ready later in the loading process, the VLM may *associate short histories with* `not_ready` *rather than learning from actual GUI changes*. This correlation may mislead the model when GUI loading speeds vary across interactions and devices.

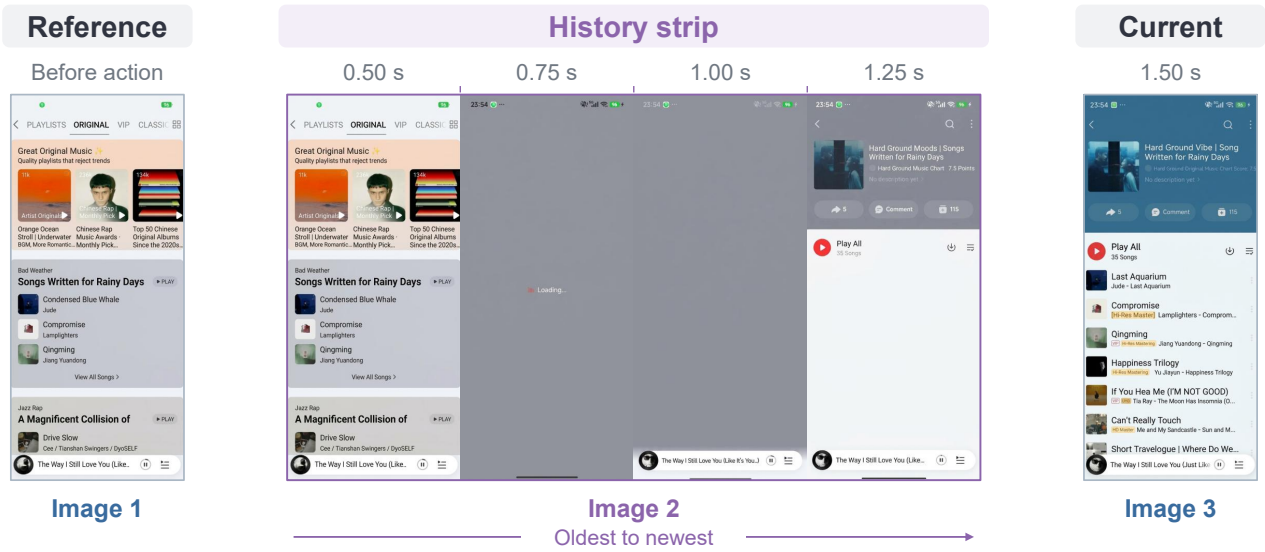



Fig. 5. A recorded NetEase Cloud Music interaction illustrating the pre-interaction reference, a chronological strip of four historical screenshots selected by the sliding window, and the current screenshot. Times are relative to action completion; image display heights are normalized for readability.

To break this coupling, we introduce *temporal resampling*, which varies observation times and historical frame spacing while *selecting history length independently of observation time*. Specifically, we draw on the variability of GUI response speeds across devices and network conditions. A similar visual progression may unfold quickly on one device and slowly on another, suggesting that temporal compression or stretching can approximate variations in response speed. For an observer sampling at a fixed interval, these variations change how much visual progress is visible between successive screenshots. We can construct such temporal variations from a recorded interaction by selecting screenshots at different temporal spacings.

For each action, target label, and history length $N \in \{0, \ldots, K\}$, we generate candidate examples in two steps and collect them in a set $C$. First, to cover the early, middle, and late stages of each phase, we divide the corresponding label interval into three equal-duration segments. We cycle through these segments, drawing a current observation time $t$ uniformly within each selected segment, as shown in Figure 6(a). Second, when $N > 0$, we partition the preceding interval $(t_0, t)$ into $N$ equal-duration strata and draw one historical frame uniformly from each stratum, as illustrated in Figure 6(b). When $N = 0$, the history is omitted. By specifying $N$ independently of $t$, this process synthesizes short or empty histories for `ready` observations and full histories for `not_ready` observations while preserving chronological order.

Finally, we select a subset $C^\star \subseteq C$ to supplement $D_G$ toward balanced coverage across the four groups formed by pairing short ($N < K$) or full ($N = K$) histories with `ready` or `not_ready`. Within the short-history groups, selection also targets balanced coverage of individual history lengths for each label. Algorithm 1 summarizes this procedure. We then fine-tune the pretrained VLM on the combined training set $D = D_G \cup C^\star$ by minimizing the cross-entropy loss over the target label $y(t)$.

**Algorithm 1:** Temporal Resampling

**Require:** Regularly sampled examples $D_G$; recordings $S$; history limit $K$
**Ensure:** Supplemented training set $D$

$C \leftarrow \varnothing$
**for** each action $s \in S$ **do**
  Read $t_0, t_R, t_E$ and the reference from $s$
  **for** each label $y$ and $N \in \{0, \ldots, K\}$ **do**
    Split the interval for $y$ into three equal segments
    Draw times $T$ by cycling through the segments, sampling uniformly within each selected segment
    **for** each selected current time $t \in T$ **do**
      $H \leftarrow \varnothing$
      **if** $N > 0$ **then**
        Divide $(t_0, t)$ into $N$ ordered strata
        $H \leftarrow$ one uniform draw per stratum
      **end if**
      Form $x$ from the reference, frames at $H$, and $I(t)$
      $C \leftarrow C \cup \{(x, y(t))\}$
    **end for**
  **end for**
**end for**
Select $C^\star \subseteq C$ toward balanced coverage of the four history-length and label groups, and individual lengths within short-history groups
**return** $D_G \cup C^\star$

## D. Integration with GUI Agents

To support integration with diverse agents and applications, we design WAITLENS as a non-intrusive, agent-agnostic

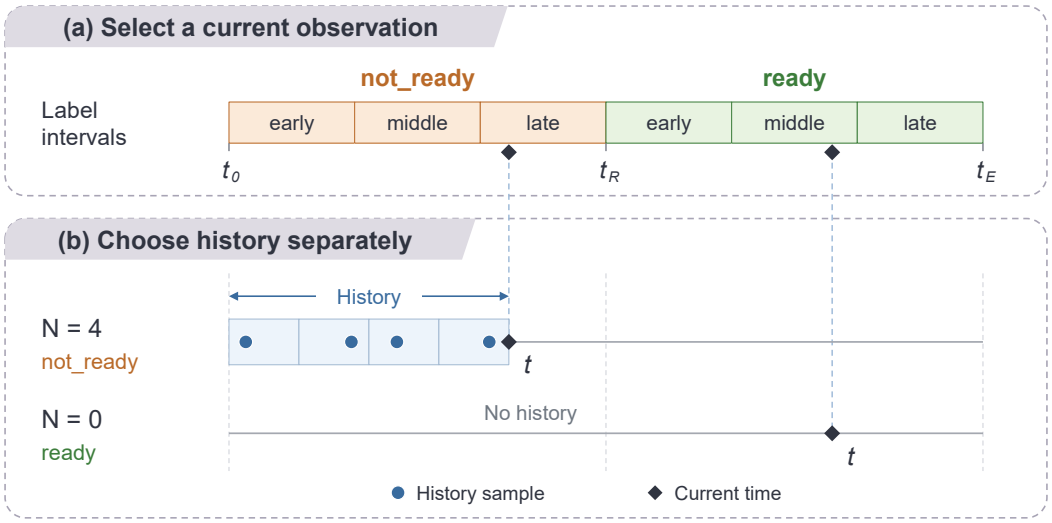



Fig. 6. Temporal resampling. (a) Each label interval is divided into three temporal segments from which current observation times are drawn. (b) Given a selected current time $t$, history is sampled from equal-duration strata of $(t_0, t)$, illustrated with $N = 4$ and $N = 0$.

plugin. Specifically, WAITLENS integrates into screenshot-based GUI agents by replacing the fixed waiting period between action execution and the next observation, as shown in the upper of Figure 4. Upon action completion at $t_0$, the agent provides WAITLENS with the pre-interaction screenshot, screenshot access, and its original waiting budget $W$. The plugin determines when to return the next screenshot without modifying the agent's reasoning or action-execution logic. Moreover, it operates entirely on screenshots, requiring no application instrumentation, source-code modification, or view-hierarchy access.

During online waiting, WAITLENS samples screenshots at interval $\Delta$ and maintains a history of up to $K$ screenshots. The waiting budget $W$ sets a deadline at $t_0 + W$. At each check, the fine-tuned VLM predicts whether the current interface is `ready` using the temporal visual input described in Section III-B. To reduce prediction fluctuations, WAITLENS supports a configurable threshold $n$ that requires $n$ consecutive `ready` predictions before returning. Once this condition is met, WAITLENS ends waiting and returns the latest checked screenshot to the agent. Otherwise, it continues checking until the original waiting budget $W$ expires. If no `ready` prediction is obtained by then, WAITLENS returns the latest available screenshot. This allows the agent to resume earlier when the GUI is ready while retaining the original waiting budget as a fallback.

## IV. EVALUATION

We evaluate WAITLENS using GUI interaction recordings collected from Meituan's industrial testing pipeline. Our evaluation examines GUI readiness prediction and waiting-policy performance through offline replay. Specifically, we address the following research questions (RQs):

1) **RQ1:** How accurately and efficiently does WAITLENS predict GUI readiness, and how does temporal resampling affect its performance?
2) **RQ2:** How effectively can WAITLENS reduce extra waiting time while minimizing premature returns in industrial GUI testing?

### A. Experimental Setup

*1) Datasets:* For **RQ1**, we use 102 annotated interaction recordings covering 12 testing tasks from Meituan. We split the tasks into training, validation, and test sets with 7, 2, and 3 tasks, respectively. All recordings and derived observations from the same task are assigned to the same set to avoid data leakage. Following Section III-C, we construct 994 training, 700 validation, and 848 test observations using regular sampling and temporal resampling. To evaluate readiness prediction under both regular sampling and varying observation intervals, the test set contains 424 regularly sampled and 424 temporally resampled observations.

For **RQ2**, we use a larger dataset collected from Meituan's daily regression testing pipeline to evaluate waiting policies through offline replay. It contains 1,101 recordings and $M = 3{,}868$ post-action GUI response sequences, each serving as one evaluation unit. Of these, 512 recordings (1,670 interactions) are captured directly from device screens, while 589 recordings (2,198 interactions) are captured by external cameras using physical test rigs. We geometrically correct the external recordings to align their frames with the screen viewport. Figure 7 shows examples of the two recording types. The earliest Ready time for each interaction was annotated by Meituan testing personnel, providing ground truth for evaluating premature returns and post-Ready waiting. The RQ1 and RQ2 datasets contain no overlapping recordings.

*2) Baselines and Variants:* For **RQ1**, we compare WAITLENS with four baselines: pretrained Qwen3-VL-2B-Instruct,

Qwen3.8-Max [37], AdaT [14], and a variant of WAITLENS trained without temporal resampling. The Qwen3-VL-2B-Instruct evaluates the effect of fine-tuning, while Qwen3.8-Max serves as a larger general-purpose VLM baseline. Both VLMs receive the same temporal visual input as WAITLENS. For AdaT, we use its released MobileNet classifier with the current screenshot, mapping its fully and partially rendered predictions to `ready` and `not_ready`, respectively. We also evaluate an ablation variant of WAITLENS. It is trained without temporal resampling, using only $D_G$, whereas the complete model is trained on $D_G \cup C^\star$.

For **RQ2**, we compare WAITLENS with AdaT and fixed waiting intervals of $w \in \{1, 2, 3, 5, 10, 20\}$ s. The 20 second duration is the default configuration in Meituan's testing pipeline. Both WAITLENS and AdaT use the same maximum waiting budget of $W = 20$ s and follow the return and deadline rules in Section III-D. We set $n = 2$ to reduce prediction fluctuations and the risk of premature returns.

*3) Metrics:* For **RQ1**, we treat predicting whether the GUI is Ready as a binary classification task with `ready` as the positive class. We report Accuracy, false positive rate (FPR), and false negative rate (FNR). A false positive classifies an unready interface as `ready`, which may cause a premature return. A false negative classifies a Ready interface as `not_ready`, leading to additional checks and longer waiting times. Since the waiting policy may lead to repeated model calls, we also measure the latency of each model call. In detail, we report the median and 95th percentile (P95) model-call latency. Both are measured from submitting the prepared visual input to receiving the prediction, including network latency for API-based models.

For **RQ2**, we use three metrics to evaluate each waiting policy. First, the premature return rate (PRR) is the proportion of interactions in which the returned screenshot was captured before the GUI reached Ready. Second, actual waiting time measures the elapsed time from action completion until the policy returns a screenshot, including checking overhead. Third, extra waiting after Ready measures the time spent waiting after the GUI reaches Ready and is set to zero for premature returns. We report PRR and the mean and P95 of both time metrics.

*4) Implementation and Configuration:* We use Qwen3-VL-2B-Instruct [24] as the backbone of WAITLENS, with a default sampling interval of $\Delta = 0.25$ s and a maximum history length of $K = 4$. The reference and current screenshots are resized to a longest edge of 768 pixels. Each historical screenshot is resized to a longest edge of 384 pixels, and the historical screenshots are concatenated horizontally. All screenshots retain their original aspect ratios.

We fine-tune the model using LoRA with rank 16 and $\alpha = 32$, and AdamW with a learning rate of $5 \times 10^{-5}$. All VLM-based predictors use the same prompt to output `ready` or `not_ready`. Local VLM inference runs on a single NVIDIA A100 GPU with bfloat16 precision, a batch size of one, and greedy decoding. Qwen3.8-Max is accessed through its API with temperature set to zero.

### B. RQ1: Ready State Prediction and Overhead

As shown in Table I, WAITLENS achieves the highest accuracy of 86.08% and the lowest FPR of 24.59% among the evaluated methods. Compared with Qwen3-VL-2B and Qwen3.8-Max, WAITLENS improves accuracy by 31.95 and 26.53 percentage points, respectively. Both general-purpose VLMs exhibit high FPRs (75.41% and 74.94%), frequently predicting `ready` for unready interfaces. AdaT, using its released checkpoint, predicts `ready` for all test observations, resulting in an FPR of 100%. These results highlight the benefit of task-specific fine-tuning for predicting GUI readiness.

The ablation results further show the contribution of temporal resampling. Incorporating temporal resampling improves overall accuracy by 2.12 percentage points and reduces FPR from 29.47% to 24.59%, with only a minor increase in FNR from 2.16% to 2.88%. This trade-off is important for waiting policies because false positives can cause premature returns, while false negatives lead to additional checks and longer waiting times. These results suggest that temporal resampling helps reduce premature `ready` predictions with only a small increase in FNR.

Finally, inference latency is also important because the waiting policy may invoke the model repeatedly. As shown in Table I, WAITLENS achieves a median latency of 0.243 s and a P95 of 0.786 s per call. In comparison, Qwen3.8-Max has a median latency of 1.607 s, approximately 6.6 times that of WAITLENS, and a P95 of 2.839 s when accessed through its API. WAITLENS achieves higher prediction accuracy and lower model-call latency than Qwen3.8-Max.

### C. RQ2: Waiting Policies in Industrial Scenarios

Beyond evaluating individual predictions in RQ1, RQ2 examines how effectively WAITLENS controls the post-action waiting process in industrial testing. In live online testing, network fluctuations introduce unpredictable variations in interface loading times, which can cause the agent's subsequent execution trajectory to diverge and make it difficult to fairly compare different waiting policies. Therefore, to conduct a strictly controlled comparison, we collect 3,868 interaction response processes from Meituan's daily regression pipeline. We execute all waiting policies on the exact same production response streams and incorporate the measured duration of every check into the waiting timeline.

As shown in Table II, Meituan's default 20 s fixed waiting strategy achieves a near-zero PRR of 0.13%. But this conservative strategy incurs an average of 17.957 s of extra waiting after the interface reaches Ready. In comparison, WAITLENS reduces mean actual waiting from 20.000 s to 2.222 s (P95 of 4.023 s). It also reduces mean extra waiting by 94.98%, from 17.957 s to 0.901 s (P95 of 1.558 s). However, these savings come with a higher PRR of 15.64%, indicating that premature returns remain a challenge. Only 2.53% of interactions reach the 20-second fallback deadline.

Furthermore, comparisons with shorter fixed waiting durations demonstrate that simply reducing a static threshold

TABLE I
READY-STATE CLASSIFICATION AND MODEL-CALL LATENCY ON THE MEITUAN TEST SET OF 848 OBSERVATIONS. BOLDFACE HIGHLIGHTS THE BEST ACCURACY, FPR, AND LATENCY RESULTS.

| Model or Variant | Ready-State Classification (%) | | | Model-Call Latency (s) | |
|---|---|---|---|---|---|
| | Accuracy ↑ | FPR ↓ | FNR ↓ | Median ↓ | P95 ↓ |
| AdaT | 49.17 | 100.00 | 0.00 | **0.026** | **0.056** |
| Qwen3-VL-2B | 54.13 | 75.41 | 15.35 | 0.153 | 0.286 |
| Qwen3.8-Max | 59.55 | 74.94 | 4.80 | 1.607 | 2.839 |
| WAITLENS (w/o Resampling) | 83.96 | 29.47 | 2.16 | 0.232 | 0.734 |
| WAITLENS | **86.08** | **24.59** | 2.88 | 0.243 | 0.786 |

TABLE II
WAITING-POLICY RESULTS FOR 3,868 INTERACTIONS FROM MEITUAN'S DAILY REGRESSION PIPELINE (PRR: PREMATURE RETURN RATE).

| Waiting Policy | PRR (%) ↓ | Actual Waiting Time (s) | | Post-Ready Waiting (s) | |
|---|---|---|---|---|---|
| | | Mean ↓ | P95 ↓ | Mean ↓ | P95 ↓ |
| Fixed 20 s | 0.13 | 20.000 | 20.000 | 17.957 | 19.384 |
| Fixed 10 s | 2.25 | 10.000 | 10.000 | 8.027 | 9.384 |
| Fixed 5 s | 8.04 | 5.000 | 5.000 | 3.268 | 4.384 |
| Fixed 3 s | 13.16 | 3.000 | 3.000 | 1.472 | 2.384 |
| Fixed 2 s | 23.60 | 2.000 | 2.000 | 0.640 | 1.384 |
| Fixed 1 s | 71.79 | 1.000 | 1.000 | 0.067 | 0.384 |
| AdaT | 90.80 | 1.864 | 20.000 | 1.464 | 18.650 |
| **WAITLENS** (all) | 15.64 | 2.222 | 4.023 | 0.901 | 1.558 |
| Internal ($n = 1{,}670$) | 12.87 | 2.328 | 3.805 | 0.940 | 1.483 |
| External ($n = 2{,}198$) | 17.74 | 2.142 | 4.163 | 0.871 | 1.638 |

cannot replicate the efficiency-reliability balance of WAITLENS. Because interface response times vary widely across interactions, any single static duration faces a rigid trade-off. Conservative settings of 10 s and 5 s keep PRR relatively low (2.25% and 8.04%, respectively). However, they still incur 8.027 s and 3.268 s of mean extra waiting, respectively, more than three to eight times that of WAITLENS.

Conversely, shortening the fixed wait to 2 s or 1 s to approach WAITLENS's roughly 2 s waiting time causes PRR to surge to 23.60% and 71.79%, respectively. Such aggressive thresholds frequently return unready interfaces to the agent. Even compared with a 3 s fixed wait (13.16% PRR), WAITLENS reduces mean actual waiting by 25.9% and mean extra waiting by 38.8%, with a slightly higher PRR of 15.64%. However, the suitability of a 3 s threshold can only be assessed retrospectively from the observed response times. In online testing, a fixed threshold selected from historical data may not generalize to changing device and network conditions. In contrast, WAITLENS dynamically determines when to end waiting based on the observed GUI state, without requiring a predefined threshold tailored to the response time distribution.

Compared with the prior learning-based baseline AdaT, WAITLENS achieves substantially higher reliability in returning ready interfaces. In industrial testing, an adaptive policy offers little practical value if it reduces waiting time mainly by returning unready interfaces. AdaT's PRR reaches 90.80%, meaning that more than nine out of ten interactions return unready interfaces. By leveraging temporal VLM reasoning, WAITLENS reduces PRR by 75.16 percentage points to 15.64%.

Finally, we evaluate WAITLENS across internal screen recordings and external camera recordings, achieving PRRs of 12.87% and 17.74%, respectively. Despite not being trained on external camera recordings, WAITLENS maintains comparable performance on this unseen recording modality. These results demonstrate its potential to generalize across different recording setups in industrial GUI testing.

### *D. Case Study*

In RQ2, WAITLENS returns screenshots before the GUI reaches Ready in 15.64% of the evaluated interactions. To better understand these failures, we analyze the premature returns in the Meituan dataset and identify two typical patterns. Figure 7 compares screenshots judged Ready by WAITLENS with later screenshots from the same interactions.

The first failure pattern involves incomplete content on an otherwise structured page. Specifically, the page structure is already visible, but the main content is still missing. In Case 01 of Figure 7(a), the merchant header is already visible, but the package list and purchase buttons have not appeared.

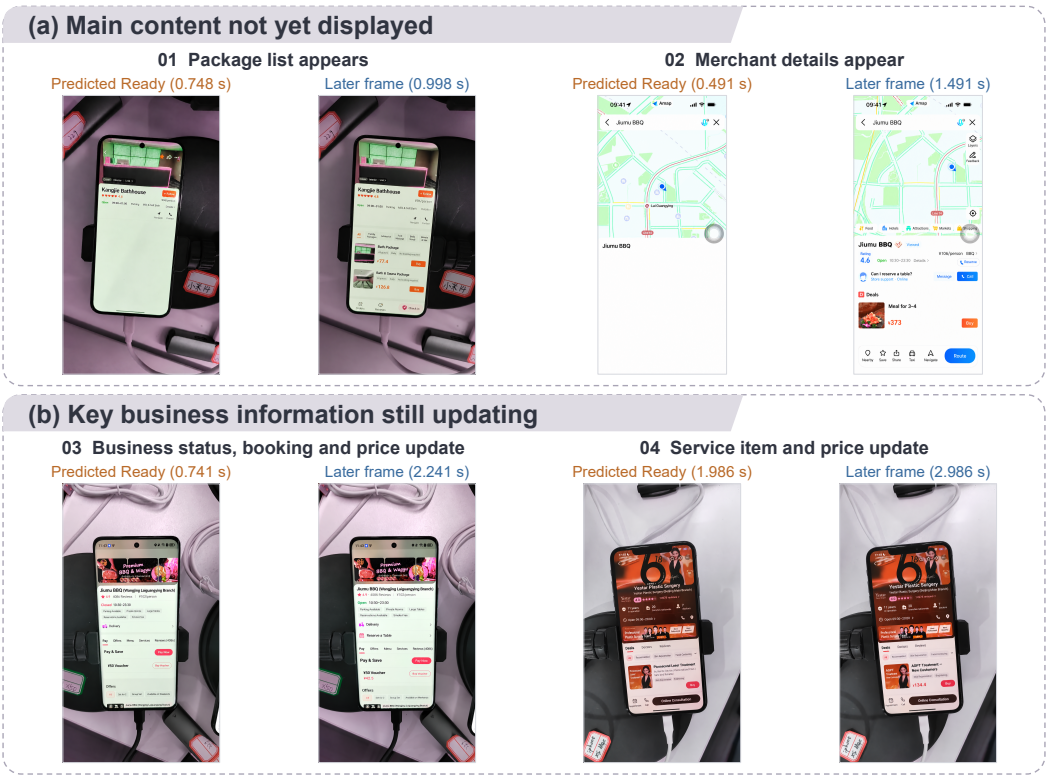



Fig. 7. Examples of premature returns. Each pair compares a screenshot judged Ready by WAITLENS with a later screenshot from the same interaction.

Similarly, in Case 02, the map and merchant title are displayed, while the merchant details and deals appear later. Without explicit loading indicators, the missing content may resemble intentionally empty regions, causing the model to predict `ready` prematurely.

The second pattern involves asynchronous updates to content that initially appears complete. In Case 03 of Figure 7(b), the merchant status, booking entry, and voucher price change after WAITLENS predicts `ready`. Similarly, in Case 04, the first service item changes even though the service list and purchase buttons are already visible. Unlike the first pattern, these pages contain seemingly complete content, making it difficult to determine whether further updates are expected.

Together, these cases illustrate the limitations of relying solely on visual observations to determine GUI readiness. Without knowing the preceding action and testing goal, WAITLENS may fail to identify missing content or detect updates to important business information. A promising direction for future work is to provide the model with the preceding action and testing goal. This context could help the model identify expected content and relevant updates before deciding whether the GUI is Ready.

## V. DISCUSSION

### A. Limitations

First, WAITLENS relies solely on visual evidence. Intermediate and completed GUI states may appear similar even with temporal context, leading to premature observations or unnecessary waiting. The deadline fallback limits waiting when no `ready` result is available, but the screenshot returned at the deadline may still precede Ready.

Second, the efficiency benefit of adaptive waiting depends on the waiting time it can avoid relative to its checking cost. Screenshot sampling determines when visual evidence becomes available, while input preparation and model execution introduce further delay. For fast interactions with short fixed waits, these costs can leave little room for improvement. Model selection and sampling frequency should therefore account for the host's waiting budget and deployment latency alongside prediction accuracy.

### B. Threats to Validity

Annotators may differ in identifying the earliest ready time for progressively rendered interfaces. We mitigate this variation through an explicit criterion for the Ready state, two independent annotators, and a third reviewer who resolves disagreements. Because multiple observations are derived from each recording, all derived observations remain within the source recording's partition. We further partition Meituan

recordings by task group to limit overlap between training and evaluation.

Moreover, our findings may not generalize to all applications, interaction types, and device conditions. Video replay can compare premature returns and post-Ready waiting across policies using the same recorded response sequence. Because this sequence remains fixed, replay does not capture how changes in observation timing affect subsequent agent actions. Establishing effects on task success and overall execution efficiency requires closed-loop agent evaluation.

## VI. Related Work

### A. GUI Testing Agents

Recent advances in LLMs have facilitated the development of autonomous GUI testing agents [1]–[6], [8], [38], reducing the reliance on manually written test scripts [29], [30]. For example, GPTDroid [1] leverages LLMs to guide functionality-aware GUI exploration, while DroidAgent [2] autonomously generates testing goals and interacts with apps to achieve them. More recently, AUITestAgent [4] enables automated GUI interaction and functional verification based on natural-language test requirements. Despite their different testing objectives, these agents generally follow an iterative process: observing GUI states, reasoning about subsequent actions, and interacting with apps. Existing studies primarily focus on improving test generation, exploration, and verification. In contrast, our work investigates an orthogonal problem: reducing the waiting overhead between consecutive interactions without compromising testing effectiveness.

### B. Waiting Strategies for GUI Testing

Prior studies have investigated various waiting strategies to improve the effectiveness and efficiency of automated GUI testing. In web GUI testing, WEFix [39] analyzes browser UI changes to automatically generate explicit wait conditions, while Zhang et al. [16] identify expected DOM changes between consecutive actions to determine when tests should proceed. Moura et al. [40] further empirically evaluated different waiting mechanisms in Cypress tests, highlighting the importance of an efficient waiting strategy.

For mobile GUI testing, AdaT [14] employs a CNN-based model to infer GUI rendering states from screenshots and dynamically schedule subsequent test actions. Its follow-up work, WeReplay [15], applies rendering inference to improve the efficiency of record-and-replay testing in WeChat. More recently, TurboDroid [17] combines static dependency analysis and runtime monitoring to determine when individual GUI widgets are ready for interaction. In contrast, our work focuses on adaptive waiting for GUI testing agents in industrial practice, where diverse GUI states and heterogeneous devices place greater demands on the generalizability of waiting strategies.

### C. VLMs for GUI Understanding

Recent advances in VLMs have enabled increasingly sophisticated GUI understanding. Some studies primarily focus on understanding GUI content from static screenshots. For example, Ferret-UI [19] enhances the ability of VLMs to recognize, ground, and reason about GUI elements, while Ferret-UI 2 [20] extends these capabilities across different platforms. Similarly, SeeClick [21] improves visual GUI grounding to support automated interaction.

More recently, researchers have explored VLMs for understanding dynamic GUI states. Liang et al. [41] investigate VLMs' ability to interpret UI animations, revealing remaining challenges in understanding their meanings. PerFrame [42] employs a VLM to identify loading start and end frames based on recorded screencasts, enabling GUI loading performance monitoring. In contrast, our work predicts at runtime. It focuses on runtime GUI rendering estimation to guide adaptive waiting for GUI testing agents.

## VII. Conclusion

This paper shows that fixed waiting in GUI testing agents struggles to accommodate substantial variations in interface response times. Our industrial motivation study reveals that over 60% of the configured waiting time is spent idling after the interface is already ready for the next observation, resulting in substantial redundant waiting. To address this, we present WaitLens, a lightweight VLM-based adaptive waiting approach for GUI testing agents. By examining temporal GUI changes, WaitLens predicts whether the GUI is Ready and determines when to end waiting. Experiments on annotated GUI interaction recordings show that WaitLens achieves 86.08% accuracy, compared with 54.13% for its pretrained backbone, with a median latency of 0.243 s per model call. Furthermore, our industrial evaluation within Meituan's production testing pipeline demonstrates WaitLens's real-life usability in substantially reducing redundant waiting while maintaining high observation reliability.

## References


[1] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 100:1–100:13.

[2] J. Yoon, R. Feldt, and S. Yoo, "Intent-driven mobile GUI testing with autonomous large language model agents," in *Proceedings of the 17th IEEE Conference on Software Testing, Verification and Validation*, 2024, pp. 129–139.

[3] D. Ran, H. Wang, Z. Song, M. Wu, Y. Cao, Y. Zhang, W. Yang, and T. Xie, "Guardian: A runtime framework for LLM-based UI exploration," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 958–970.

[4] Y. Hu, X. Wang, Y. Wang, Y. Zhang, S. Guo, C. Chen, X. Wang, and Y. Zhou, "AUITestAgent: Automatic requirements oriented GUI function testing," *CoRR*, vol. abs/2407.09018, 2024.

[5] M. Chen, Z. Liu, C. Chen, J. Wang, Y. Xue, B. Wu, Y. Huang, L. Wu, and Q. Wang, "Beyond static GUI agent: Evolving LLM-based GUI testing via dynamic memory," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, 2025, pp. 1603–1615.

[6] Q. Kong, Z. Lv, Y. Xiong, J. Sun, T. Su, D. Wang, L. Li, X. Yang, and G. Huo, "ProphetAgent: Automatically synthesizing GUI tests from test cases in natural language for mobile apps," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '25, 2025, pp. 174–179.

[7] S. Feng, C. Du, H. Liu, Q. Wang, Z. Lv, G. Huo, X. Yang, and C. Chen, "Agent for user: Testing multi-user interactive features in TikTok," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice*. IEEE, 2025, pp. 57–68.

[8] X. Wang, Y. Wang, Y. Hu, Y. Zhang, H. Jin, S. Guo, J. Yuan, and Y. Zhou, "From redundancy to efficiency: Exploiting shared UI interactions towards efficient LLM-based testing," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, 2025, pp. 3438–3449.

[9] Q. Wu, P. Gao, W. Liu, and J. Luan, "BacktrackAgent: Enhancing GUI agent with error detection and backtracking mechanism," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2025, pp. 4250–4272.

[10] P. Wu, S. Ma, B. Wang, J. Yu, L. Lu, and Z. Liu, "GUI-Reflection: Empowering multimodal GUI models with self-reflection behavior," in *Advances in Neural Information Processing Systems 38*, 2025.

[11] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, "AppAgent: Multimodal agents as smartphone users," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 70:1–70:20.

[12] J. Wang, H. Xu, J. Ye, M. Yan, W. Shen, J. Zhang, F. Huang, and J. Sang, "Mobile-Agent: Autonomous multi-modal mobile device agent with visual perception," *CoRR*, vol. abs/2401.16158, 2024.

[13] J. Wang, H. Xu, H. Jia, X. Zhang, M. Yan, W. Shen, J. Zhang, F. Huang, and J. Sang, "Mobile-Agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 2686–2710.

[14] S. Feng, M. Xie, and C. Chen, "Efficiency matters: Speeding up automated testing with GUI rendering inference," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 906–918.

[15] S. Feng, H. Lu, T. Xiong, Y. Deng, and C. Chen, "Towards efficient record and replay: A case study in WeChat," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1681–1692.

[16] H. Zhang, L. Liao, Z. Ding, W. Shang, N. Narula, C. Sporea, A. Toma, and S. Sajedi, "Towards a robust waiting strategy for web GUI testing for an industrial software system," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2065–2076.

[17] X. Li, X. Yin, C. Ni, X. Xin, C. Sun, J. Wang, and X. Yang, "Breaking waiting: Accelerating android GUI testing via widget readiness analysis," *ACM Transactions on Software Engineering and Methodology*, 2026, just Accepted.

[18] G. Baechler, S. Sunkara, M. Wang, F. Zubach, H. Mansoor, V. Etter, V. Cărbune, J. Lin, J. Chen, and A. Sharma, "ScreenAI: A vision-language model for UI and infographics understanding," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 3058–3068.

[19] K. You, H. Zhang, E. Schoop, F. Weers, A. Swearngin, J. Nichols, Y. Yang, and Z. Gan, "Ferret-UI: Grounded mobile UI understanding with multimodal LLMs," in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science, vol. 15122. Springer, 2024, pp. 240–255.

[20] Z. Li, K. You, H. Zhang, D. Feng, H. Agrawal, X. Li, M. P. S. Moorthy, J. Nichols, Y. Yang, and Z. Gan, "Ferret-UI 2: Mastering universal user interface understanding across platforms," in *The Thirteenth International Conference on Learning Representations*, 2025.

[21] K. Cheng, Q. Sun, Y. Chu, F. Xu, Y. Li, J. Zhang, and Z. Wu, "SeeClick: Harnessing GUI grounding for advanced visual GUI agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 9313–9332.

[22] D. Chen, Y. Huang, S. Wu, J. Tang, H. Zhou, Q. Zhang, Z. He, Y. Bai, C. Gao, L. Chen, Y. Li, C. Wang, Y. Yu, T. Zhou, Z. Li, Y. Gui, Y. Wan, P. Zhou, J. Gao, and L. Sun, "GUI-World: A video benchmark and dataset for multimodal GUI-oriented understanding," in *The Thirteenth International Conference on Learning Representations*, 2025.

[23] K. Q. Lin, L. Li, D. Gao, Q. Wu, M. Yan, Z. Yang, L. Wang, and M. Z. Shou, "VideoGUI: A benchmark for GUI automation from instructional videos," in *Advances in Neural Information Processing Systems 37*, 2024.

[24] Qwen Team, "Qwen3-VL technical report," *CoRR*, vol. abs/2511.21631, 2025.

[25] Meituan, "Announcement of the results for the three months ended september 30, 2025," Meituan, Tech. Rep., Nov. 2025, official earnings announcement.

[26] T. Su, G. Meng, Y. Chen, K. Wu, W. Yang, Y. Yao, G. Pu, Y. Liu, and Z. Su, "Guided, stochastic model-based GUI testing of Android apps," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. ACM, 2017, pp. 245–256.

[27] Y. Li, Z. Yang, Y. Guo, and X. Chen, "DroidBot: A lightweight UI-guided test input generator for android," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, 2017, pp. 23–26.

[28] Z. Lv, C. Peng, Z. Zhang, T. Su, K. Liu, and P. Yang, "Fastbot2: Reusable automated model-based GUI testing for Android enhanced by reinforcement learning," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2022, pp. 1–5.

[29] G. Hu, L. Zhu, and J. Yang, "AppFlow: Using machine learning to synthesize robust, reusable UI tests," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2018, pp. 269–282.

[30] J. Guo, S. Li, J.-G. Lou, Z. Yang, and T. Liu, "SARA: Self-replay augmented record and replay for android in industrial cases," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 90–100.

[31] International Software Testing Qualifications Board, *Certified Tester Specialist: Mobile Application Testing Foundation Level Syllabus*, 2019, version 2019.

[32] Amazon Web Services, *What Is AWS Device Farm?*, Amazon Web Services, 2026, AWS Device Farm Developer Guide.

[33] Google, *Firebase Test Lab*, Google, 2026, firebase Documentation.

[34] C. Rawles, S. Clinckemaillie, Y. Chang, J. Waltz, G. Lau, M. Fair, A. Li, W. E. Bishop, W. Li, F. Campbell-Ajala, D. K. Toyama, R. J. Berry, D. Tyamagundlu, T. P. Lillicrap, and O. Riva, "AndroidWorld: A dynamic benchmarking environment for autonomous agents," in *The Thirteenth International Conference on Learning Representations*, 2025.

[35] C. Zhang, H. Huang, C. Ni, J. Mu, S. Qin, S. He, L. Wang, F. Yang, P. Zhao, C. Du, L. Li, Y. Kang, Z. Jiang, S. Zheng, R. Wang, J. Qian, M. Ma, J.-G. Lou, Q. Lin, S. Rajmohan, and D. Zhang, "UFO2: The desktop AgentOS," *CoRR*, vol. abs/2504.14603, 2025.

[36] D. Zhang, B. Rama, J. Ni, S. He, F. Zhao, K. Chen, A. Chen, and J. Cao, "LiteWebAgent: The open-source suite for VLM-based web-agent applications," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*. Association for Computational Linguistics, 2025, pp. 449–455.

[37] Qwen Team, "Qwen3.8-Max: A new bar for coding and cowork," Aug. 2026, official model announcement.

[38] Z. Liu, C. Li, C. Chen, J. Wang, M. Chen, B. Wu, Y. Wang, J. Hu, and Q. Wang, "Seeing is believing: Vision-driven non-crash functional bug detection for mobile apps," *IEEE Transactions on Software Engineering*, vol. 51, no. 12, pp. 3452–3466, 2025.

[39] X. Liu, Z. Song, W. Fang, W. Yang, and W. Wang, "WEFix: Intelligent automatic generation of explicit waits for efficient web end-to-end flaky tests," in *Proceedings of the ACM Web Conference 2024*, 2024, pp. 3043–3052.

[40] T. S. de Moura, E. L. G. Alves, R. L. S. Felipe, C. de Souza Baptista, I. R. da Silva Neto, and H. F. de Figueirêdo, "Addressing the synchronization challenge in cypress end-to-end tests," in *Anais do XXXVIII Simpósio Brasileiro de Engenharia de Software (SBES 2024)*, 2024, pp. 92–102.

[41] C. Liang, X. Jiang, N. Deng, E. Adar, and A. Guo, "Beyond screenshots: Evaluating VLMs' understanding of UI animations," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 12 907–12 927.

[42] J. Liu, S. Guo, Y. Hu, Y. Zhang, H. Jin, J. Yuan, Y. Zhou, and X. Wang, "PerFrame: Monitoring GUI loading performance in mobile apps via semantic distinguish," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*, 2026, pp. 440–450.